



UNIVERSITY MANAGEMENT SYSTEM

Café Management Part



Submitted by:

Name: Wasima Khanom Munira

ID: 2022-1-60-287

Course: CSE423 Section: 01

Submitted to:

Name: Ahmed Adnan

Designation: Lecturer, CSE Department, EWU.

1. Café Management: (Solid Principles)

Overview of SOLID Principles

The uploaded files demonstrate excellent application of SOLID principles throughout the codebase. Below is a detailed analysis of each principle with justification and code snippets.

1. Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change.

Applied In:

DrinkSelection.java

```
java
// Each enum has single responsibility
enum DrinkCategory {
    SOFT_DRINKS("Soft Drinks", 1),
    COFFEE("Coffee", 2),
    JUICE_PLANT("Juice or Plant Drinks", 3);
// Only defines drink categories
}

// Separate interfaces for distinct responsibilities
interface DrinkMenuDisplay {
    void display();
}

interface DrinkInputReader {
    int readInt();
    void clearBuffer();
    void close();
}
// Single responsibility: Handle console input
class DrinkConsoleInputReader implements DrinkInputReader {
    private final Scanner scanner;
// Only handles input reading
}
```

MyCoffee.java

```
java
// ENUM only defines coffee items
enum CoffeeItem {
    ESPRESSO("Espresso", 120, 1);
// Only defines items, prices, codes
}

// Single responsibility: Display menu
interface CoffeeMenuDisplayer {
    void displayMenu();
}

// Single responsibility: Handle input
interface CoffeeInputReader {
    int readInt();
    int readQuantity();
}

// Single responsibility: Process orders
class CoffeeOrderProcessor {
    private final List<CoffeeOrder> orders;
// Only processes orders
}

// Single responsibility: Write bills
interface CoffeeBillWriter {
    void writeHeader(String regNumber, Date date) throws IOException;
    void writeOrder(CoffeeOrder order) throws IOException;
}
```

MyFinalBill.java

```
java
// Each class has one reason to change
interface StudentDataService { /* Gets student data */ }
interface BillReader { /* Reads bills */ }
interface BillPrinter { /* Prints bills */ }
```

```

interface BillSaver { /* Saves bills */ }
interface TempFileCleaner { /* Cleans temp files */ }

class BalanceValidator { /* Only validates balance */ }
class FileBillReader implements BillReader { /* Only reads files */ }
class ConsoleBillPrinter implements BillPrinter { /* Only prints to console */ }

```

Why Apply SRP?

- Easier maintenance and debugging
- Changes in one responsibility don't affect others
- Code is more testable
- Reduces complexity

2. Open/Closed Principle (OCP)

Definition: Classes should be open for extension but closed for modification.

Applied In:

DrinkSelection.java - Strategy Pattern

```

java
// Strategy Pattern - Open for new handlers, closed for modification
interface DrinkCategoryHandler {
    void handle(String regNumber);
    DrinkCategory getCategory();
}

// Can add new handler without modifying existing code
class SoftDrinksHandler extends BaseDrinkHandler {
    @Override
    public void handle(String regNumber) {
        int total = MySoftDrinks.SoftDrinkbill(regNumber);
    }
}

// New handler can be added easily
class CoffeeHandler extends BaseDrinkHandler {

```

```

@Override
public void handle(String regNumber) {
    int total = MyCoffee.CoffeeBill(regNumber);
}
}

// Factory allows adding new handlers without modification
class DefaultDrinkHandlerFactory implements DrinkHandlerFactory {
    private final Map<Integer, DrinkCategoryHandler> handlers;

    public DefaultDrinkHandlerFactory() {
        registerHandler(new SoftDrinksHandler());
        registerHandler(new CoffeeHandler());
        registerHandler(new JuicePlantHandler());
        // Can register new handlers without changing existing code
    }
}

```

MyFastFood.java

```

java
// Enum - can add new items without modifying existing logic
enum FastFoodItem {
    BURGER("Burger", 80),
    ZINGER_BURGER("Zinger Burger", 250),
    // Can add new items without changing the enum structure
    // DEAL_D("Deal D", 900); - Just add new constant
}

// fromChoice method doesn't need modification for new items
static FastFoodItem fromChoice(int choice) {
    if (choice < 1 || choice > values().length) return null;
    return values()[choice - 1]; // Works with any number of items
}

```

MyDesiFood.java

```

java
// Adding new desi dish doesn't require code changes
enum DesiDish {

```

```
HALEEM("Haleem", 120),
ROGAN_GOSHT("Rogan Gosht", 250),
// New dishes: KABAB("Seekh Kabab", 180) - Just add
}
```

Why Apply OCP?

- Reduces risk when adding new features
- Promotes code reuse
- Makes system more maintainable
- Supports plugin-like architecture

3. Liskov Substitution Principle (LSP)

Definition: Derived classes must be substitutable for their base classes.

Applied In:

DrinkSelection.java

```
java
// Base handler can be replaced by any derived handler
abstract class BaseDrinkHandler implements DrinkCategoryHandler {
    protected final String handlerName;

    protected BaseDrinkHandler(String handlerName) {
        this.handlerName = handlerName;
    }

    @Override
    public String getHandlerName() {
        return handlerName;
    }
}

// All derived handlers are substitutable
class SoftDrinksHandler extends BaseDrinkHandler {
    public SoftDrinksHandler() {
        super("Soft Drinks Handler");
    }
}
```

```
// Implements handle() correctly  
}
```

```
class CoffeeHandler extends BaseDrinkHandler {  
    public CoffeeHandler() {  
        super("Coffee Handler");  
    }  
// Implements handle() correctly  
}
```

```
// Enhanced handlers also substitutable  
class EnhancedSoftDrinksHandler extends SoftDrinksHandler  
implements EnhancedDrinkHandler {  
    @Override  
    public int handleWithTotal(String regNumber) {  
// Works as drop-in replacement  
        return super.handleWithTotal(regNumber);  
    }  
}
```

MyFinalBill.java

```
java  
// Any BillReader implementation can be used interchangeably  
interface BillReader {  
    BillReadingResult readAllBills();  
}  
  
class FileBillReader implements BillReader {  
    @Override  
    public BillReadingResult readAllBills() {  
// Implementation  
    }  
}  
  
// Can create DatabaseBillReader that also implements BillReader  
// and it will work without modifying the FinalBillService
```

Why Apply LSP?

- Ensures reliable inheritance hierarchies
- Makes code more predictable
- Facilitates polymorphism
- Prevents unexpected behavior

4. Interface Segregation Principle (ISP)

Definition: Clients should not be forced to depend on interfaces they do not use.

Applied In:

DrinkSelection.java

```
java
// Small, focused interfaces instead of one large interface
interface DrinkMenuDisplayer {
    void display(); // Only display method
}

interface DrinkInputReader {
    int readInt();
    void clearBuffer();
    void close();
}

interface DrinkCategoryHandler {
    void handle(String regNumber);
    DrinkCategory getCategory();
    String getHandlerName();
}

// Separate enhanced interface for those who need total
interface EnhancedDrinkHandler extends DrinkCategoryHandler {
    int handleWithTotal(String regNumber);
}

// Observer interface with specific methods
```

```
interface DrinkSelectionObserver {  
    void onDrinkCategorySelected(DrinkCategory category, String regNumber, int total);  
    void onDrinkMenuExited();  
}
```

MyCoffee.java

```
java  
// Segregated interfaces  
interface CoffeeMenuDisplay {  
    void displayMenu(); // Only menu display  
}  
  
interface CoffeeInputReader {  
    int readInt();  
    int readPositiveInt();  
    int readQuantity();  
    Optional<Integer> readChoice();  
    void close();  
}  
  
interface CoffeeBillWriter {  
    void writeHeader(String regNumber, Date date) throws IOException;  
    void writeOrder(CoffeeOrder order) throws IOException;  
    void writeTotal(int total) throws IOException;  
    void close() throws IOException;  
}
```

MyFinalBill.java

```
java  
// Each interface has single, focused purpose  
interface StudentDataService {  
    StudentInfo getStudentInfo(String regNumber) throws IOException;  
}  
  
interface BillReader {  
    BillReadingResult readAllBills();  
}
```

```
interface BillPrinter {  
    void printBill(StudentInfo student, BillReadingResult billData, int remainingBalance);  
}
```

```
interface BillSaver {  
    void saveBill(StudentInfo student, BillReadingResult billData) throws IOException;  
}
```

```
interface TempFileCleaner {  
    void clearTempFiles();  
    void clearTempFiles(List<String> errors);  
}
```

Why Apply ISP?

- Reduces coupling between components
- Makes interfaces more reusable
- Prevents "fat interfaces"
- Easier to implement and maintain

5. Dependency Inversion Principle (DIP)

Definition: Depend on abstractions, not concretions.

Applied In:

DrinkSelection.java

```
java  
// Service depends on abstractions, not concrete implementations  
class DrinkSelectionService {  
    private final DrinkMenuDisplay menuDisplay;    // Abstraction  
    private final DrinkInputReader inputReader;    // Abstraction  
    private final DrinkHandlerFactory handlerFactory; // Abstraction  
  
    public DrinkSelectionService(DrinkMenuDisplay menuDisplay,  
        DrinkInputReader inputReader,  
        DrinkHandlerFactory handlerFactory) {  
        this.menuDisplay = Objects.requireNonNull(menuDisplay);  
    }  
}
```

```

this.inputReader = Objects.requireNonNull(inputReader);
this.handlerFactory = Objects.requireNonNull(handlerFactory);
}
}

// Factory that creates abstractions
interface DrinkHandlerFactory {
    Optional<DrinkCategoryHandler> getHandler(int choice);
    Optional<EnhancedDrinkHandler> getEnhancedHandler(int choice);
}

// Builder pattern for flexible construction
class DrinkSelectionServiceBuilder {
    private DrinkMenuDisplay menuDisplay;
    private DrinkInputReader inputReader;
    private DrinkHandlerFactory handlerFactory;

    public DrinkSelectionServiceBuilder withDefaultMenuDisplay() {
        this.menuDisplay = new DrinkCategoryConsoleMenuDisplay();
        return this;
    }

    public DrinkSelectionService build() {
        // Returns abstraction, not concrete implementation
        return new DrinkSelectionService(menuDisplay, inputReader, handlerFactory);
    }
}

```

MyFinalBill.java

```

java
// High-level module depends on abstractions
class FinalBillService {
    private final StudentDataService studentService; // Abstraction
    private final BillReader billReader;           // Abstraction
    private final BalanceValidator balanceValidator; // Abstraction
    private final BillPrinter billPrinter;         // Abstraction
    private final BillSaver billSaver;             // Abstraction
    private final TempFileCleaner tempFileCleaner; // Abstraction
}

```

```

// Constructor depends on interfaces, not concrete classes
public FinalBillService(StudentDataService studentService,
    BillReader billReader,
    BalanceValidator balanceValidator,
    BillPrinter billPrinter,
    BillSaver billSaver,
    TempFileCleaner tempFileCleaner) {
    // All parameters are abstractions
}

// Builder allows injecting different implementations
class FinalBillServiceBuilder {
    public FinalBillServiceBuilder withStudentService(StudentDataService service) {
        this.studentService = service;
        return this;
    }

    public FinalBillServiceBuilder withBillReader(BillReader reader) {
        this.billReader = reader;
        return this;
    }
}

```

MyJuiceOrPlantDrink.java

```

java
// Service depends on abstractions
class JuicePlantBillGenerationService {
    private final JuicePlantMenuService menuService;    // Abstraction
    private final JuicePlantInputReader inputReader;    // Abstraction
    private final JuicePlantOrderProcessor orderProcessor; // Abstraction
    private final JuicePlantBillWriter personalWriter;    // Abstraction
    private final JuicePlantBillWriter sharedWriter;    // Abstraction
}

// Factory creates concrete implementations but returns abstractions
class JuicePlantServiceFactory {

```

```

public static JuicePlantBillGenerationService createService(String regNumber) {
    JuicePlantMenuService menuService = new FileBasedJuicePlantMenuService();
    JuicePlantInputReader inputReader = new JuicePlantConsoleInputReader();
    // Returns abstraction
    return new JuicePlantBillGenerationService(/* ... */);
}
}

```

Why Apply DIP?

- Increases flexibility and reusability
- Makes testing easier (mock implementations)
- Reduces coupling between modules
- Supports dependency injection

Before Refactoring:

```

-----
Date: Fri Mar 13 19:14:42 GMT+06:00 2026
Name: MARZIA HOSSAN
Student ID: SP19-BCS-127
-----Your Bill Is-----

Items          Qty  Price
Zinger Burger   2    500
Zinger Burger   1    250
Aloo Ka Paratha  1    100
-----
Total Bill:      850 TK
Remaining Balance: 1150 TK
-----
Remaining Balance: 1150 TK

```

After Refactoring:

```

-----
Date: Fri Mar 13 19:10:19 GMT+06:00 2026
Name: QASIM ALI
Student ID: FA18-BCS-131
-----Your Bill Is-----

Items          Qty  Price
Zinger Burger   2    500
Haleem          1    120
-----
Total Bill:      620 TK
Remaining Balance: 1380 TK
-----
Remaining Balance: 1380 TK

```

2. Café Management: (Refactor)

Selection Method:

1. Long Method / Complex Method

Smell: chooseMethod() method has multiple responsibilities

- ☐ Displaying menu
- ☐ Taking input
- ☐ Validating input
- ☐ Calling food/drink modules
- ☐ Handling errors

```
public static void chooseMethod(String regNumber) throws Exception {
    final int INITIAL_AMOUNT = 2000;
    while (true) {
        printMainMenu();
        try {
            System.out.print("Enter Your Choice: ");
            option = Integer.parseInt(input.nextLine().trim());
            System.out.println();
            switch (option) {
                case 1: foodMenu(regNumber); break;
                case 2: drinkMenu(regNumber); break;
                case 3: int remainingBalance = MyFinalBill.finalBill(regNumber, INITIAL_AMOUNT);
                    System.out.println("Remaining Balance: " + remainingBalance + " TK");
                    break;
                case 4: clearTemporaryFiles(); System.out.println("Returning to Previous Menu...");
                    return;
                case 5: clearTemporaryFiles(); System.out.println("Thank you for using Cafe Managemen
                    nt System!"); System.exit(0);
                default: System.out.println("Please select correct option");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid Input - Please enter a number");
        } catch (Exception e) {
            System.out.println("Error: " + e.getMessage()); e.printStackTrace();
        }
    }
}
```

Problems with long methods:

1. Hard to understand
2. Difficult to test
3. Difficult to maintain
4. High risk of bugs
5. Low reusability

Refactoring Remedy: EXTRACT METHOD

```

public static void chooseMethod(String regNumber) {
    while (true) {
        printMainMenu();
        int option = readIntInput("Enter Your Choice: ");

        switch (option) {
            case 1 -> foodMenu(regNumber);
            case 2 -> drinkMenu(regNumber);
            case 3 -> generateBill(regNumber);
            case 4 -> {
                clearTemporaryFiles();
                System.out.println("Returning to Previous Menu...");
                return;
            }
            case 5 -> {
                clearTemporaryFiles();
                System.out.println("*****");
                System.out.println("Thank you for using Cafe Management System!");
                System.exit(0);
            }
        }
    }
}

```

```

        case 5 -> {
            clearTemporaryFiles();
            System.out.println("*****");
            System.out.println("Thank you for using Cafe Management System!");
            System.exit(0);
        }
        default -> System.out.println("Please select a correct option");
    }
}
}

```

Benefit

- Code readable
- Responsibility separated
- Maintainability improved

2. Duplicated Code (Menu Display)

Smell: Multiple menu display methods with similar structure

Several classes had **similar menu display code** like:

- FastFood menu
- DesiFood menu
- Coffee menu
- Drinks menu

Code Snippet (Smell):

```
// In foodMenu()
System.out.println("\n==== FOOD CATEGORIES =====");
System.out.println("1. Fast Food");
System.out.println("2. Desi Food");
System.out.println("0. Back to Main Menu");
System.out.println("=====");

// In drinkMenu()
System.out.println("\n==== DRINK CATEGORIES =====");
System.out.println("1. Soft Drinks");
System.out.println("2. Coffee");
System.out.println("3. Juice or Plant Drinks");
System.out.println("0. Back to Main Menu");
System.out.println("=====");
```

Refactoring Remedy: EXTRACT METHOD + PARAMETERIZE METHOD

```
private static void printMenu(String title, String[] options) {
    System.out.println("\n==== " + title + " =====");
    for (int i = 0; i < options.length; i++) {
        System.out.printf("%d. %s\n", i + 1, options[i]);
    }
    System.out.println("=====");
}

// Usage in foodMenu
String[] foodOptions = {"Fast Food", "Desi Food", "Back to Main Menu"};
printMenu("FOOD CATEGORIES", foodOptions);
```

Benefit

- reusable code
- maintainable
- less duplication

3. Primitive Obsession & Duplicated Input Handling

Smell: Multiple try-catch blocks for input parsing

Code Snippet (Smell):

```
try {
    System.out.print("Enter Your Choice: ");
    option = Integer.parseInt(input.nextLine().trim());
    System.out.println();

    switch (option) {
        case 1:
            // Food selection
            foodMenu(regNumber);
            break;

        case 2:
            // Drink selection
            drinkMenu(regNumber);
            break;

        case 3:
            // Bill Generate - Use MyFinalBill instead of custom display
            int remainingBalance = MyFinalBill.finalBill(regNumber, INITIAL_AMOUNT);
            System.out.println("Remaining Balance: " + remainingBalance + " TK");
            break;

        case 4:
            clearTemporaryFiles();
            System.out.println("Returning to Previous Menu...");
            return;

        case 5:
            clearTemporaryFiles();
            System.out.println("*****");
            System.out.println("Thank you for using Cafe Management System!");
            System.exit(0);
    }
}

// Food Menu
private static void foodMenu(String regNumber) {
    while (true) {
        System.out.println("\n---- FOOD CATEGORIES ----");
        System.out.println("1. Fast Food");
        System.out.println("2. Desi Food");
        System.out.println("0. Back to Main Menu");
        System.out.println("-----");
        System.out.print("Enter Your Choice: ");

        try {
            int choice = Integer.parseInt(input.nextLine().trim());

            if (choice == 0) {
                System.out.println("Returning to Main Menu...");
                System.out.println("*****");
                break;
            }

            switch (choice) {
                case 1:
                    MyFastFood fastFood = new MyFastFood();
                    fastFood.FastFoodBill();
                    break;

                case 2:
                    MyDesiFood desiFood = new MyDesiFood();
                    desiFood.DesiFoodBill();
                    break;

                default:
                    System.out.println("Invalid option!");
            }
        } catch (NumberFormatException e) {
            System.out.println("Invalid input!");
        }
    }
}
```

Refactoring Remedy: EXTRACT METHOD + INTRODUCE LOCAL EXTENSION

```
private static int readIntInput(String prompt) {
    while (true) {
        System.out.print(prompt);
        String inputStr = input.nextLine().trim();
        try {
            return Integer.parseInt(inputStr);
        } catch (NumberFormatException e) {
            System.out.println("Invalid input! Please enter a number.");
        }
    }
}
```

4. Magic Numbers/Literals

Smell: Hardcoded file paths and values

Code Snippet (Smell):

```

private static void clearTemporaryFiles() throws IOException {
    String[] billFiles = {
        "cafe/CafeBills/BillFastfood.txt",
        "cafe/CafeBills/BillDesiFood.txt",
        "cafe/CafeBills/BillSoftDrinks.txt",
        "cafe/CafeBills/BillCoffee.txt",
        "cafe/CafeBills/BillJuiceOrPlant.txt"
    };

    for (String filePath : billFiles) {
        File tempFile = new File(filePath);
        if (tempFile.exists()) {
            try (PrintWriter writer = new PrintWriter(tempFile)) {
                writer.print(s: "");
            }
        }
    }

    System.out.println(x: "\n✅ Temporary files cleared");
}

```

Problem

Hardcoded values cause:

- poor readability
- difficult maintenance

Refactoring Remedy: REPLACE MAGIC NUMBER WITH CONSTANT

```

private static final Scanner input = new Scanner(System.in); 1 usage
private static final String[] TEMP_BILL_FILES = { 1 usage
    "cafe/CafeBills/BillFastfood.txt",
    "cafe/CafeBills/BillDesiFood.txt",
    "cafe/CafeBills/BillSoftDrinks.txt",
    "cafe/CafeBills/BillCoffee.txt",
    "cafe/CafeBills/BillJuiceOrPlant.txt"
};

```

5. Empty Catch Block

Smell: Exception handling without proper logging or user feedback

Code Snippet (Smell):

```

for (String filePath : billFiles) {
    File tempFile = new File(filePath);
    if (tempFile.exists()) {
        try (PrintWriter writer = new PrintWriter(tempFile)) {
            writer.print(s: "");
        }
    }
}

```

Problem

The catch block is empty.

This hides runtime errors.

Refactoring Remedy: ADD EXCEPTION HANDLING

```
try (PrintWriter writer = new PrintWriter(file)) {
    writer.print("");
} catch (IOException e) {
    System.out.println("Error clearing file: " + filePath);
}
```

Benefits

- better debugging
- improved reliability

6. Dead Code / Unused Variables

Smell: Class-level variables that are never used

Code Snippet (Smell):

```
static Scanner input = new Scanner(System.in);
static int option; // Not used outside chooseMethod
static java.util.Date date = new java.util.Date(); // Never used
static String name; // Never used
static List<String> allOrderedItems = new ArrayList<>(); // Never used
static int grandTotal = 0; // Never used
```

Problem

Unused variables increase code complexity.

Refactoring Remedy: REMOVE DEAD CODE

```
private static final Scanner input = new Scanner(System.in);
// Only kept the necessary variables, removed unused ones
```

Benefits

- cleaner code
- improved readability

7. Throw Generic Exception

Smell: Method throws too broad exception type

Code Snippet (Smell):

```
public static void chooseMethod(String regNumber) throws Exception {
```

Problem

Too generic.

Refactoring Remedy: REMOVE THROWS CLAUSE + HANDLE INTERNALLY

```
public static void chooseMethod(String regNumber) {  
    // No throws clause - handled internally  
}
```

8. Mixed Abstraction Levels

Smell: Bill generation logic mixed with menu navigation

Code Snippet (Smell):

```
case 3:  
    int remainingBalance = MyFinalBill.finalBill(regNumber, INITIAL_AMOUNT);  
    System.out.println("Remaining Balance: " + remainingBalance + " TK");  
    break;
```

Problem

High-level logic mixed with low-level operations.

Refactoring Remedy: EXTRACT METHOD

```
private static void generateBill(String regNumber) {  
    try {  
        int remainingBalance = MyFinalBill.finalBill(regNumber, INITIAL_BALANCE);  
        System.out.println("Remaining Balance: " + remainingBalance + " TK");  
    } catch (Exception e) {  
        System.out.println("Error generating bill: " + e.getMessage());  
        e.printStackTrace();  
    }  
}
```

9. Feature Envy

Smell: Using static variables unnecessarily

Code Snippet (Smell):

```
static int option; // Used locally but declared globally
```

Problem

Methods depend on external variables too much.

Refactoring Remedy: REPLACE GLOBAL VARIABLE WITH LOCAL VARIABLE

```
// Removed global 'option' variable
// Now using local variable in chooseMethod()
int option = readIntInput("Enter Your Choice: ");
```

Benefits

- reduces coupling
- safer code

10. Inconsistent Error Handling

Smell: Different error handling approaches in different methods

Code Snippet (Smell):

```
catch (NumberFormatException e) {
    System.out.println("Invalid Input - Please enter a number");
} catch (Exception e) {
    System.out.println("Error: " + e.getMessage());
    e.printStackTrace();
}
```

Problem

Different error handling styles in different methods.

Refactoring Remedy: UNIFY ERROR HANDLING

```
private static int readIntInput(String prompt) {
    while (true) {
        System.out.print(prompt);
        String inputStr = input.nextLine().trim();
        try {
            return Integer.parseInt(inputStr);
        } catch (NumberFormatException e) {
            System.out.println("Invalid input! Please enter a number.");
        }
    }
}
```

My Fast Food:

1. Dead Code (Dispensable)

Location: Unused DATE variable

Code Snippet (Smell):

```
private static final java.util.Date DATE = new java.util.Date();  
// This variable is NEVER used anywhere in the class
```

Problem

The variable was declared but **never used anywhere in the program**.

Dead code causes:

- unnecessary complexity
- confusion for developers
- larger codebase without purpose

Remedy: REMOVE DEAD CODE

```
// Removed completely from refactored code  
// (Not present in refactored version)
```

Benefit

- cleaner class
- easier readability
- better maintainability

2. Long Method (Bloater)

Location: FastfoodBill() method is too long (40+ lines)

Problem

The method performed multiple responsibilities:

- displaying the menu
- taking user input
- validating quantity
- calculating price
- printing bill

Code Snippet (Smell):

```

public int FastfoodBill() {
    List<OrderedItem> orders = new ArrayList<>();
    int sessionTotal = 0;

    System.out.println(x: "\n===== FAST FOOD MENU =====");
    for (int i = 0; i < FastFoodItem.values().length; i++) {
        FastFoodItem item = FastFoodItem.values()[i];
        System.out.printf(format: "%d. %s (%d Rs)\n", i+1, item.displayName, item.price);
    }
    System.out.println(x: "0. Nothing / Go Back");
    System.out.println(x: "=====");

    while (true) {
        System.out.print(s: "\nEnter Your Choice (0 to finish): ");
        int choice;
        try {
            choice = Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println(x: "Invalid input!");
            continue;
        }

        if (choice == 0) {
            break;
        }
    }
}

```

```

        FastFoodItem selected = FastFoodItem.fromChoice(choice);
        if (selected == null) {
            System.out.println(x: "Invalid choice!");
            continue;
        }

        System.out.print(s: "Enter quantity: ");
        int quantity;
        try {
            quantity = Integer.parseInt(scanner.nextLine().trim());
            if (quantity <= 0) {
                System.out.println(x: "Quantity must be positive!");
                continue;
            }
        } catch (NumberFormatException e) {
            System.out.println(x: "Invalid quantity!");
            continue;
        }

        OrderedItem order = new OrderedItem(selected, quantity);
        orders.add(order);
        sessionTotal += order.getTotalPrice();

        System.out.printf(format: "✓ Added: %s x %d = %d TK\n", selected.displayName, quantity, order.getTotalPrice());
        System.out.println("Current total: " + sessionTotal + " TK");
    }

    if (!orders.isEmpty()) {
        saveToFile(orders);
        System.out.println("\n✅ Fast Food Bill Generated. Total: " + sessionTotal + " TK");
    }

    return sessionTotal;
}

```

Remedy: EXTRACT METHOD

```

public int FastfoodBill() {
    List<OrderedItem> orders = new ArrayList<>();
    int sessionTotal = 0;

    displayMenu(); // EXTRACTED

    while (true) {
        int choice = readInt("Enter Your Choice (0 to finish): ");
        if (choice == 0) break;

        FastFoodItem selected = FastFoodItem.fromChoice(choice);
        if (selected == null) {
            System.out.println("Invalid choice!");
            continue;
        }

        int quantity = readInt("Enter quantity: ");
        if (quantity <= 0) {
            System.out.println("Quantity must be positive!");
            continue;
        }
    }
}

```

```

        OrderedItem order = new OrderedItem(selected, quantity);
        orders.add(order);
        sessionTotal += order.getTotalPrice();

        System.out.printf("✓ Added: %s x %d = %d TK\n", selected.displayName, quantity, order.getTotalPrice());
        System.out.println("Current total: " + sessionTotal + " TK");
    }

    if (!orders.isEmpty()) {
        BillFileService.saveOrders(orders); // EXTRACTED to separate class
        System.out.println("\n🟢 Fast Food Bill Generated. Total: " + sessionTotal + " TK");
    }

    return sessionTotal;
}

```

Benefits

- each method has one responsibility
- easier debugging
- better readability
- reusable logic

3. Duplicate Code (Dispensable)

Location: Multiple try-catch blocks for input handling

Code Snippet (Smell):

```

// First try-catch for choice
try {
    choice = Integer.parseInt(scanner.nextLine().trim());
} catch (NumberFormatException e) {
    System.out.println("Invalid input!");
    continue;
}

// Second try-catch for quantity (SAME PATTERN)
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) {
        System.out.println("Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println("Invalid quantity!");
    continue;
}

```

Problem

The same input validation code was repeated across multiple classes.

Duplicate code leads to:

- harder maintenance
- higher bug risk
- violation of **DRY (Don't Repeat Yourself)** principle

Remedy: EXTRACT METHOD

```
private int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            return Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Invalid input!"); // Consistent message
        }
    }
}
```

Benefits

- centralized validation
- reusable method
- consistent behavior

4. Magic String (Primitive Obsession)

Location: Hardcoded file path

Code Snippet (Smell):

```
try (FileWriter writer = new FileWriter("cafe/CafeBills/BillFastfood.txt", true)) {
    // Magic string directly in code
}
```

Problem

Hardcoded strings make the program difficult to maintain.

If the path changes, it must be updated everywhere.

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {
    public static final String BILL_DIR = "cafe/CafeBills";
    public static final String BILL_FILE = BILL_DIR + "/BillFastfood.txt";
    // Used throughout the class
}
```

Benefits

- easier updates
- improved readability
- avoids repeated literals

5. Magic Strings for Menu (Primitive Obsession)

Location: Menu headers and footers hardcoded

Code Snippet (Smell):

```
System.out.println(x: "\n==== FAST FOOD MENU =====");
for (int i = 0; i < FastFoodItem.values().length; i++) {
    FastFoodItem item = FastFoodItem.values()[i];
    System.out.printf(format: "%d. %s (%d Rs)\n", i+1, item.displayName, item.price);
}
System.out.println(x: "0. Nothing / Go Back");
System.out.println(x: "=====");
```

Problem

Two responsibilities mixed together:

- input parsing
- validation

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants { 4 usages  MuniraRoma
    public static final String BILL_DIR = "cafe/CafeBills"; 2 usages
    public static final String BILL_FILE = BILL_DIR + "/BillFastfood.txt"; 1 usage
    public static final String MENU_HEADER = "\n==== FAST FOOD MENU ====="; 1 usage
    public static final String MENU_FOOTER = "0. Nothing / Go Back\n=====";
}
```

Benefit

- clearer logic
- easier testing
- modular validation

6. Mixed Validation Logic (Primitive Obsession)

Location: Quantity validation inside try-catch

Code Snippet (Smell):

```

System.out.print(s: "Enter quantity: ");
int quantity;
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) {
        System.out.println(x: "Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println(x: "Invalid quantity!");
    continue;
}
}

```

Remedy: SEPARATE CONCERNS

```

int quantity = readInt("Enter quantity: ");
if (quantity <= 0) { // Validation separate
    System.out.println("Quantity must be positive!");
    continue;
}

```

7. Lazy Class / Incomplete Abstraction (Dispensable)

Location: File operations not properly abstracted

Code Snippet (Smell):

```

private void saveToFile(List<OrderedItem> orders) {
    try {
        new File("cafe/CafeBills").mkdirs();

        try (FileWriter writer = new FileWriter("cafe/CafeBills/BillFastfood.txt", true)) {
            for (OrderedItem order : orders) {
                writer.write(order.getBillLine() + "\n");
            }
        }
    } catch (IOException e) {
        System.out.println("Error saving: " + e.getMessage());
    }
}

```

Problem

File operations were **not properly abstracted**.

Remedy: EXTRACT CLASS (Better Abstraction)

```
private static class BillFileService {
    static void saveOrders(List<OrderedItem> orders) {
        try {
            new File(Constants.BILL_DIR).mkdirs();
            try (FileWriter writer = new FileWriter(Constants.BILL_FILE, true)) {
                for (OrderedItem order : orders) {
                    writer.write(order.getBillLine() + "\n");
                }
            }
        } catch (IOException e) {
            System.out.println("Error saving: " + e.getMessage());
        }
    }
}
```

Benefit

- better abstraction
- reusable file handling
- separation of responsibilities

My Desi Food:

1. Dead Code (Dispensable)

Location: Unused DATE variable

Code Snippet (Smell):

```
private static final java.util.Date DATE = new java.util.Date();
// This variable is NEVER used anywhere in the class
```

Problem

The variable was declared but **never used anywhere in the program**.

Dead code causes:

- unnecessary complexity
- confusion for developers
- larger codebase without purpose

Remedy: REMOVE DEAD CODE

```
// Removed completely from refactored code  
// (Not present in refactored version)
```

Benefit

- cleaner class design
- improved readability
- reduced unnecessary code

2. Long Method (Bloater)

Location: DesifoodBill() method is too long (40+ lines)

Code Snippet (Smell):

```
public int DesifoodBill() {  
    List<OrderedDish> orders = new ArrayList<>();  
    int sessionTotal = 0;  
  
    System.out.println(x: "\n==== DESI FOOD MENU =====");  
    for (int i = 0; i < DesiDish.values().length; i++) {  
        DesiDish dish = DesiDish.values()[i];  
        System.out.printf(format: "%d. %s (%d Rs)\n", i+1, dish.displayName, dish.price);  
    }  
    System.out.println(x: "0. Nothing / Go Back");  
    System.out.println(x: "=====");  
  
    while (true) {  
        System.out.print(s: "\nEnter Your Choice (0 to finish): ");  
        int choice;  
        try {  
            choice = Integer.parseInt(scanner.nextLine().trim());  
        } catch (NumberFormatException e) {  
            System.out.println(x: "Invalid input!");  
            continue;  
        }  
  
        if (choice == 0) {  
            break;  
        }  
  
        DesiDish selected = DesiDish.fromChoice(choice);  
        if (selected == null) {  
            System.out.println(x: "Invalid choice!");  
            continue;  
        }  
    }  
}
```

```
        System.out.print(s: "Enter quantity: ");  
        int quantity;  
        try {  
            quantity = Integer.parseInt(scanner.nextLine().trim());  
            if (quantity <= 0) {  
                System.out.println(x: "Quantity must be positive!");  
                continue;  
            }  
        } catch (NumberFormatException e) {  
            System.out.println(x: "Invalid quantity!");  
            continue;  
        }  
  
        OrderedDish order = new OrderedDish(selected, quantity);  
        orders.add(order);  
        sessionTotal += order.getTotalPrice();  
  
        System.out.printf(format: "✓ Added: %s x %d = %d TK\n", selected.displayName, quantity, order.getTotalPrice());  
        System.out.println("Current total: " + sessionTotal + " TK");  
    }  
  
    // Save to file if any items were ordered  
    if (!orders.isEmpty()) {  
        saveToFile(orders);  
        System.out.println("\n✅ Desi Food Bill Generated. Total: " + sessionTotal + " TK");  
    } else {  
        System.out.println(x: "No Desi Food items ordered.");  
    }  
    return sessionTotal;  
}
```

Problem

The method performed multiple tasks such as:

- displaying the desi food menu
- reading user selection
- reading quantity

- calculating the total bill
- printing the bill

Remedy: EXTRACT METHOD

```
public int DesiFoodBill() {
    List<OrderedDish> orders = new ArrayList<>();
    int sessionTotal = 0;

    displayMenu(); // EXTRACTED

    while (true) {
        int choice = readInt("Enter Your Choice (0 to finish): ");
        if (choice == 0) break;

        DesiDish selected = DesiDish.fromChoice(choice);
        if (selected == null) {
            System.out.println("Invalid choice!");
            continue;
        }

        int quantity = readInt("Enter quantity: ");
        if (quantity <= 0) {
            System.out.println("Quantity must be positive!");
            continue;
        }
    }
}
```

```
OrderedDish order = new OrderedDish(selected, quantity);
orders.add(order);
sessionTotal += order.getTotalPrice();

System.out.printf("\n✓ Added: %s x %d = %d TK\n", selected.displayName, quantity, order.getTotalPrice());
System.out.println("Current total: " + sessionTotal + " TK");
}

if (!orders.isEmpty()) {
    BillFileService.saveOrders(orders); // EXTRACTED to separate class
    System.out.println("\n✅ Desi Food Bill Generated. Total: " + sessionTotal + " TK");
} else {
    System.out.println("No Desi Food items ordered.");
}

return sessionTotal;
}
```

Benefits

- improved readability
- easier debugging
- each method performs one task

3. Duplicate Code (Dispensable)

Location: Multiple try-catch blocks for input handling

Code Snippet (Smell):

```
// First try-catch for choice
try {
    choice = Integer.parseInt(scanner.nextLine().trim());
} catch (NumberFormatException e) {
    System.out.println("Invalid input!");
    continue;
}

// Second try-catch for quantity (SAME PATTERN)
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) {
        System.out.println("Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println("Invalid quantity!");
    continue;
}
```

Problem

The same input validation code appeared multiple times.

This causes:

- code duplication
- maintenance difficulty
- violation of the **DRY principle**
-

Remedy: EXTRACT METHOD

```
private int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            return Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Invalid input!"); // Consistent message
        }
    }
}
```

Benefits

- centralized validation logic
- reusable method
- reduced duplicate code

4. Magic String (Primitive Obsession)

Location: Hardcoded file path

Code Snippet (Smell):

```
try (FileWriter writer = new FileWriter("cafe/CafeBills/BillDesiFood.txt", true)) {
    // Magic string directly in code
}
```

Problem

- Hardcoded values make code harder to maintain.
- If the file path changes, it must be updated everywhere.

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {
    public static final String BILL_DIR = "cafe/CafeBills";
    public static final String BILL_FILE = BILL_DIR + "/BillDesiFood.txt";
    // Used throughout the class
}
```

Benefits

- easier maintenance
- better readability

5. Magic Strings for Menu (Primitive Obsession)

Location: Menu headers and footers hardcoded

Code Snippet (Smell):

```
System.out.println(x: "\n===== DESI FOOD MENU =====");
for (int i = 0; i < DesiDish.values().length; i++) {
    DesiDish dish = DesiDish.values()[i];
    System.out.printf(format: "%d. %s (%d Rs)\n", i+1, dish.displayName, dish.price);
}
System.out.println(x: "0. Nothing / Go Back");
System.out.println(x: "=====");
```

Problem

Two responsibilities mixed together:

- input parsing
- validation

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {
    public static final String MENU_HEADER = "\n===== DESI FOOD MENU =====";
    public static final String MENU_FOOTER = "0. Nothing / Go Back\n=====";
}
```

Benefit

- clearer logic
- easier testing

- modular validation

6. Mixed Validation Logic (Primitive Obsession)

Location: Quantity validation inside try-catch

Code Snippet (Smell):

```
System.out.print(s: "Enter quantity: ");
int quantity;
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) {
        System.out.println(x: "Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println(x: "Invalid quantity!");
    continue;
}
```

Remedy: SEPARATE CONCERNS

```
int quantity = readInt("Enter quantity: ");
if (quantity <= 0) { // Validation separate
    System.out.println("Quantity must be positive!");
    continue;
}
```

7. Incomplete Abstraction (Dispensable)

Location: File operations not properly abstracted

Code Snippet (Smell):

```
private void saveToFile(List<OrderedDish> orders) {
    try {
        new File("cafe/CafeBills").mkdirs();

        try (FileWriter writer = new FileWriter("cafe/CafeBills/BillDesiFood.txt", true)) {
            for (OrderedDish order : orders) {
                writer.write(order.getBillline() + "\n");
            }
        }
    } catch (IOException e) {
        System.out.println("Error saving: " + e.getMessage());
    }
}
```

Problem

File operations were **not properly abstracted**.

Remedy: EXTRACT CLASS (Better Abstraction)

```
private static class BillFileService {
    static void saveOrders(List<OrderedItem> orders) {
        try {
            new File(Constants.BILL_DIR).mkdirs();
            try (FileWriter writer = new FileWriter(Constants.BILL_FILE, true)) {
                for (OrderedItem order : orders) {
                    writer.write(order.getBillline() + "\n");
                }
            }
        } catch (IOException e) {
            System.out.println("Error saving: " + e.getMessage());
        }
    }
}
```

Benefit

- better abstraction
- reusable file handling
- separation of responsibilities

My Coffee:

1. Dead Code (Dispensable)

Location: Unused DATE variable

Code Snippet (Smell):

```
private static final java.util.Date DATE = new java.util.Date();
// This variable is NEVER used anywhere in the class
```

Problem

The variable was declared but never used.

Dead code causes:

- unnecessary complexity
- reduced readability

Remedy: REMOVE DEAD CODE

```
// Removed completely from refactored code
// (Not present in refactored version)
```

Benefit

- cleaner class design
- improved readability
- reduced unnecessary code

2. Long Method (BLOATER)

Location: CoffeeBill() method is too long (40+ lines)

Code Snippet (Smell):

```
public int CoffeeBill(String regNumber) {
    List<CoffeeOrder> orders = new ArrayList<>();
    int total = 0;

    System.out.println(x: "\n===== COFFEE MENU =====");
    for (int i = 0; i < CoffeeItem.values().length; i++) {
        CoffeeItem coffee = CoffeeItem.values()[i];
        System.out.printf(format: "%d. %s (%d Rs)\n", i+1, coffee.name, coffee.price);
    }
    System.out.println(x: "0. Nothing / Go Back");
    System.out.println(x: "=====");

    while (true) {
        System.out.print(s: "\nEnter Your Choice (0 to finish): ");
        int choice;
        try {
            choice = Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println(x: "Invalid input!");
            continue;
        }

        if (choice == 0) break;

        CoffeeItem selected = CoffeeItem.fromChoice(choice);
        if (selected == null) {
            System.out.println(x: "Invalid choice!");
            continue;
        }

        System.out.print(s: "Enter quantity: ");
        int quantity;
        try {
            quantity = Integer.parseInt(scanner.nextLine().trim());
            if (quantity <= 0) {
                System.out.println(x: "Quantity must be positive!");
                continue;
            }
        }
    }
}
```

```
// File saving (if block)
if (!orders.isEmpty()) {
    saveToFile(orders);
    System.out.println("\n✅ Coffee Bill Generated. Total: " + total + " TK");
}

return total;
}
```

Problem

The method performed multiple tasks such as:

- displaying the desi food menu
- reading user selection
- reading quantity
- calculating the total bill
- printing the bill

Remedy: EXTRACT METHOD

```
public int CoffeeBill(String regNumber) {
    List<CoffeeOrder> orders = new ArrayList<>();
    int total = 0;

    displayMenu(); // EXTRACTED

    while (true) {
        int choice = readInt("Enter Your Choice (0 to finish): ");
        if (choice == 0) break;

        CoffeeItem selected = CoffeeItem.fromChoice(choice);
        if (selected == null) {
            System.out.println("Invalid choice!");
            continue;
        }

        int quantity = readInt("Enter quantity: ");
        if (quantity <= 0) {
            System.out.println("Quantity must be positive!");
            continue;
        }

        CoffeeOrder order = new CoffeeOrder(selected, quantity);
        orders.add(order);
        total += order.getTotal();

        System.out.printf("✓ Added: %s x %d = %d TK\n", selected.displayName, quantity, order.getTotal());
        System.out.println("Current total: " + total + " TK");
    }

    if (!orders.isEmpty()) {
        BillFileService.saveOrders(orders); // EXTRACTED to separate class
        System.out.println("\n✅ Coffee Bill Generated. Total: " + total + " TK");
    }

    return total;
}
```

Benefits

- improved readability
- easier debugging
- each method performs one task

3. Duplicate Code (Dispensable)

Location: Multiple try-catch blocks for input handling

Code Snippet (Smell):

```
// First try-catch for choice
try {
    choice = Integer.parseInt(scanner.nextLine().trim());
} catch (NumberFormatException e) {
    System.out.println("Invalid input!");
    continue;
}

// Second try-catch for quantity (SAME PATTERN)
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) {
        System.out.println("Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println("Invalid quantity!");
    continue;
}
```

Problem

The same input validation code appeared multiple times.

This causes:

- code duplication
- maintenance difficulty
- violation of the DRY principle

Remedy: EXTRACT METHOD

```
private int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            return Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Invalid input!"); // Consistent message
        }
    }
}
```

Benefits

- centralized validation logic
- reusable method
- reduced duplicate code

4. Magic String (Primitive Obsession)

Location: Hardcoded file path

Code Snippet (Smell):

```
try (FileWriter writer = new FileWriter("cafe/CafeBills/BillCoffee.txt", true)) {  
    // Magic string directly in code  
}
```

Problem

- Hardcoded values make code harder to maintain.
- If the file path changes, it must be updated everywhere.

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {  
    public static final String BILL_DIR = "cafe/CafeBills";  
    public static final String BILL_FILE = BILL_DIR + "/BillCoffee.txt";  
    // Used throughout the class  
}
```

Benefits

- easier maintenance
- better readability
-

5. Magic Strings for Menu (Primitive Obsession)

Location: Menu headers and footers hardcoded

Code Snippet (Smell):

```
System.out.println(x: "\n==== COFFEE MENU =====");  
for (int i = 0; i < CoffeeItem.values().length; i++) {  
    CoffeeItem coffee = CoffeeItem.values()[i];  
    System.out.printf(format: "%d. %s (%d Rs)\n", i+1, coffee.name, coffee.price);  
}  
System.out.println(x: "0. Nothing / Go Back");  
System.out.println(x: "=====");
```

Problem

Two responsibilities mixed together:

- input parsing
- validation
-

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {
    public static final String MENU_HEADER = "\n==== COFFEE MENU =====";
    public static final String MENU_FOOTER = "0. Nothing / Go Back\n===== ";
}
```

Benefit

- clearer logic
- easier testing
- modular validation

6. Mixed Validation Logic (Primitive Obsession)

Location: Quantity validation inside try-catch

Code Snippet (Smell):

```
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) { // Validation mixed with parsing
        System.out.println("Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println("Invalid quantity!");
    continue;
}
```

Remedy: SEPARATE CONCERNS

```
int quantity = readInt("Enter quantity: ");
if (quantity <= 0) { // Validation separate
    System.out.println("Quantity must be positive!");
    continue;
}
```

7. Incomplete Abstraction (Dispensable)

Location: File operations not properly abstracted

Code Snippet (Smell):

```
private void saveToFile(List<CoffeeOrder> orders) {
    try {
        new File("cafe/CafeBills").mkdirs();

        try (FileWriter writer = new FileWriter("cafe/CafeBills/BillCoffee.txt", true)) {
            for (CoffeeOrder order : orders) {
                writer.write(order.getBillline() + "\n");
            }
        }
    } catch (IOException e) {
        System.out.println("Error saving: " + e.getMessage());
    }
}
```

Problem

File operations were **not properly abstracted**.

Remedy: EXTRACT CLASS (Better Abstraction)

```
private static class BillFileService {
    static void saveOrders(List<CoffeeOrder> orders) {
        try {
            new File(Constants.BILL_DIR).mkdirs();
            try (FileWriter writer = new FileWriter(Constants.BILL_FILE, true)) {
                for (CoffeeOrder order : orders) {
                    writer.write(order.getBillline() + "\n");
                }
            }
        } catch (IOException e) {
            System.out.println("Error saving: " + e.getMessage());
        }
    }
}
```

Benefit

- better abstraction
- reusable file handling
- separation of responsibilities

My Soft Drinks:

1. Dead Code (Dispensable)

Location: Unused DATE variable

Code Snippet (Smell):

```
private static final java.util.Date DATE = new java.util.Date();  
// This variable is NEVER used anywhere in the class
```

Problem

The variable was declared but **never used anywhere in the program**.

Dead code causes:

- unnecessary complexity
- confusion for developers
- larger codebase without purpose

Remedy: REMOVE DEAD CODE

```
// Removed completely from refactored code  
// (Not present in refactored version)
```

Benefit

- cleaner class design
- improved readability
- reduced unnecessary code
-

2. Long Method (Bloat)

Location: SoftDrinkbill() method is too long (40+ lines)

Code Snippet (Smell):

```
public int SoftDrinkbill(String regNumber) {  
    List<DrinkOrder> orders = new ArrayList<>();  
    int total = 0;  
  
    System.out.println(x: "\n==== SOFT DRINKS MENU =====");  
    for (int i = 0; i < SoftDrink.values().length; i++) {  
        SoftDrink drink = SoftDrink.values()[i];  
        System.out.printf(format: "%d. %s (%d Rs)\n", i+1, drink.name, drink.price);  
    }  
    System.out.println(x: "0. Nothing / Go Back");  
    System.out.println(x: "=====");  
  
    while (true) {  
        System.out.print(s: "\nEnter Your Choice (0 to finish): ");  
        int choice;  
        try {  
            choice = Integer.parseInt(scanner.nextLine().trim());  
        } catch (NumberFormatException e) {  
            System.out.println(x: "Invalid input!");  
            continue;  
        }  
  
        if (choice == 0) break;  
  
        SoftDrink selected = SoftDrink.fromChoice(choice);  
        if (selected == null) {  
            System.out.println(x: "Invalid choice!");  
            continue;  
        }  
  
        System.out.print(s: "Enter quantity: ");  
        int quantity;  
        try {  
            quantity = Integer.parseInt(scanner.nextLine().trim());  
            if (quantity <= 0) {  
                System.out.println(x: "Quantity must be positive!");  
            }  
        }  
    }  
}
```

```

        System.out.print(s: "Enter quantity: ");
        int quantity;
        try {
            quantity = Integer.parseInt(scanner.nextLine().trim());
            if (quantity <= 0) {
                System.out.println(x: "Quantity must be positive!");
                continue;
            }
        } catch (NumberFormatException e) {
            System.out.println(x: "Invalid quantity!");
            continue;
        }

        DrinkOrder order = new DrinkOrder(selected, quantity);
        orders.add(order);
        total += order.getTotal();

        System.out.printf(format: "✓ Added: %s x %d = %d TK\n", selected.name, quantity, order.getTotal());
        System.out.println("Current total: " + total + " TK");
    }

    if (!orders.isEmpty()) {
        saveToFile(orders);
        System.out.println("\n🟢 Soft Drinks Bill Generated. Total: " + total + " TK");
    }

    return total;
}

private void saveToFile(List<DrinkOrder> orders) {
    try {
        new File(pathname: "cafe/CafeBills").mkdirs();

        try (FileWriter writer = new FileWriter(fileName: "cafe/CafeBills/BillSoftDrinks.txt", append: true)) {
            for (DrinkOrder order : orders) {

```

Problem

The method performed multiple tasks such as:

- displaying the desi food menu
- reading user selection
- reading quantity
- calculating the total bill
- printing the bill

Remedy: EXTRACT METHOD

```

public int SoftDrinkbill(String regNumber) {
    List<SoftDrinkOrder> orders = new ArrayList<>();
    int total = 0;

    displayMenu(); // EXTRACTED

    while (true) {
        int choice = readInt("Enter Your Choice (0 to finish): ");
        if (choice == 0) break;

        SoftDrinkItem selected = SoftDrinkItem.fromChoice(choice);
        if (selected == null) {
            System.out.println("Invalid choice!");
            continue;
        }

        int quantity = readInt("Enter quantity: ");
        if (quantity <= 0) {
            System.out.println("Quantity must be positive!");
            continue;

```

```

    }

    SoftDrinkOrder order = new SoftDrinkOrder(selected, quantity);
    orders.add(order);
    total += order.getTotal();

    System.out.printf("\n Added: %s x %d = %d TK\n", selected.displayName, quantity, order.getTot
al());
    System.out.println("Current total: " + total + " TK");
}

if (!orders.isEmpty()) {
    BillFileService.saveOrders(orders); // EXTRACTED to separate class
    System.out.println("\n✅ Soft Drinks Bill Generated. Total: " + total + " TK");
}

return total;
}

```

Benefits

- improved readability
- easier debugging
- each method performs one task

3. Duplicate Code (Dispensable)

Location: Multiple try-catch blocks for input handling

Code Snippet (Smell):

```

// First try-catch for choice
try {
    choice = Integer.parseInt(scanner.nextLine().trim());
} catch (NumberFormatException e) {
    System.out.println("Invalid input!");
    continue;
}

// Second try-catch for quantity (SAME PATTERN)
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) {
        System.out.println("Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println("Invalid quantity!");
    continue;
}

```

Problem

The same input validation code appeared multiple times.

This causes:

- code duplication
- maintenance difficulty
- violation of the **DRY principle**

Remedy: EXTRACT METHOD

```
private int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            return Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Invalid input!"); // Consistent message
        }
    }
}
```

Benefits

- centralized validation logic
- reusable method
- reduced duplicate code

4. Magic String (Primitive Obsession)

Location: Hardcoded file path

Code Snippet (Smell):

```
try (FileWriter writer = new FileWriter("cafe/CafeBills/BillSoftDrinks.txt", true)) {
    // Magic string directly in code
}
```

Problem

- Hardcoded values make code harder to maintain.
- If the file path changes, it must be updated everywhere.

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {
    public static final String BILL_DIR = "cafe/CafeBills";
    public static final String BILL_FILE = BILL_DIR + "/BillSoftDrinks.txt";
    // Used throughout the class
}
```

Benefits

- easier maintenance
- better readability

5. Magic Strings for Menu (Primitive Obsession)

Location: Menu headers and footers hardcoded

Code Snippet (Smell):

```
System.out.println(x: "\n==== SOFT DRINKS MENU =====");
for (int i = 0; i < SoftDrink.values().length; i++) {
    SoftDrink drink = SoftDrink.values()[i];
    System.out.printf(format: "%d. %s (%d Rs)\n", i+1, drink.name, drink.price);
}
System.out.println(x: "0. Nothing / Go Back");
System.out.println(x: "=====");
```

Problem

Two responsibilities mixed together:

- input parsing
- validation

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {
    public static final String MENU_HEADER = "\n==== SOFT DRINKS MENU =====";
    public static final String MENU_FOOTER = "0. Nothing / Go Back\n=====";
}
```

Benefit

- clearer logic
- easier testing
- modular validation

6. Mixed Validation Logic (Primitive Obsession)

Location: Quantity validation inside try-catch

Code Snippet (Smell):

```
System.out.print(s: "Enter quantity: ");
int quantity;
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) {
        System.out.println(x: "Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println(x: "Invalid quantity!");
    continue;
}
```

Remedy: SEPARATE CONCERNS

```
int quantity = readInt("Enter quantity: ");
if (quantity <= 0) { // Validation separate
    System.out.println("Quantity must be positive!");
    continue;
}
```

7. Incomplete Abstraction (Dispensable)

Location: File operations not properly abstracted

Code Snippet (Smell):

```
private void saveToFile(List<DrinkOrder> orders) {
    try {
        new File("cafe/CafeBills").mkdirs();

        try (FileWriter writer = new FileWriter("cafe/CafeBills/BillSoftDrinks.txt", true)) {
            for (DrinkOrder order : orders) {
                writer.write(order.getBillLine() + "\n");
            }
        }
    } catch (IOException e) {
        System.out.println("Error saving: " + e.getMessage());
    }
}
```

Problem

File operations were **not properly abstracted**.

Remedy: EXTRACT CLASS (Better Abstraction)

```
private static class BillFileService {
    static void saveOrders(List<SoftDrinkOrder> orders) {
        try {
            new File(Constants.BILL_DIR).mkdirs();
            try (FileWriter writer = new FileWriter(Constants.BILL_FILE, true)) {
                for (SoftDrinkOrder order : orders) {
                    writer.write(order.getBillLine() + "\n");
                }
            }
        } catch (IOException e) {
            System.out.println("Error saving: " + e.getMessage());
        }
    }
}
```

Benefit

- better abstraction
- reusable file handling
- separation of responsibilities

My JuiceOrPlant:

1. Dead Code (Dispensable)

Location: Unused DATE variable

Code Snippet (Smell):

```
private static final java.util.Date DATE = new java.util.Date();  
// This variable is NEVER used anywhere in the class
```

Problem

The variable was declared but never used.

Dead code causes:

- unnecessary complexity
- reduced readability

Remedy: REMOVE DEAD CODE

```
// Removed completely from refactored code
```

Benefit

- cleaner class design
- improved readability
- reduced unnecessary code

2. Long Method (BLOATER)

Location: JuiceORPlantbill() method is too long (40+ lines)

Code Snippet (Smell):

```
public int JuiceORPlantbill() {  
    List<JuiceOrder> orders = new ArrayList<>();  
    int total = 0;  
  
    System.out.println(x: "\n==== JUICE / PLANT DRINK MENU =====");  
    for (int i = 0; i < JuiceItem.values().length; i++) {  
        JuiceItem juice = JuiceItem.values()[i];  
        System.out.printf(format: "%d. %s (%d Rs)\n", i+1, juice.name, juice.price);  
    }  
    System.out.println(x: "0. Nothing / Go Back");  
    System.out.println(x: "=====");  
  
    while (true) {  
        System.out.print(s: "\nEnter Your Choice (0 to finish): ");  
        int choice;  
        try {  
            choice = Integer.parseInt(scanner.nextLine().trim());  
        } catch (NumberFormatException e) {  
            System.out.println(x: "Invalid input!");  
            continue;  
        }  
  
        if (choice == 0) break;  
  
        JuiceItem selected = JuiceItem.fromChoice(choice);  
        if (selected == null) {  
            System.out.println(x: "Invalid choice!");  
            continue;  
        }  
  
        System.out.print(s: "Enter quantity: ");  
        int quantity;  
        try {  
            quantity = Integer.parseInt(scanner.nextLine().trim());  
            if (quantity <= 0) {  
                System.out.println(x: "Invalid quantity!");  
                continue;  
            }  
            JuiceOrder order = new JuiceOrder(selected, quantity);  
            orders.add(order);  
            total += order.getTotal();  
  
            System.out.printf(format: "\n✓ Added: %s x %d = %d TK\n", selected.name, quantity, order.getTotal());  
            System.out.println("Current total: " + total + " TK");  
        }  
  
        if (!orders.isEmpty()) {  
            saveToFile(orders);  
            System.out.println("\n✅ Juice/Plant Drinks Bill Generated. Total: " + total + " TK");  
        }  
  
        return total;  
    }  
  
    private void saveToFile(List<JuiceOrder> orders) {  
        try {  
            new File(pathname: "cafe/cafeBills").mkdirs();  
  
            try (FileWriter writer = new FileWriter(filename: "cafe/cafeBills/BillJuiceOrPlant.txt", append: true)) {  
                for (JuiceOrder order : orders) {  
                    writer.write(order.getBillLine() + "\n");  
                }  
            }  
        } catch (IOException e) {  
            System.out.println("Error saving: " + e.getMessage());  
        }  
    }  
}
```

Problem

The method performed multiple tasks such as:

- displaying the desi food menu
- reading user selection
- reading quantity
- calculating the total bill
- printing the bill

Remedy: EXTRACT METHOD

```
public int JuiceORPlantbill() {
    List<JuiceOrder> orders = new ArrayList<>();
    int total = 0;

    displayMenu(); // EXTRACTED

    while (true) {
        int choice = readInt("Enter Your Choice (0 to finish): ");
        if (choice == 0) break;

        JuiceItem selected = JuiceItem.fromChoice(choice);
        if (selected == null) {
            System.out.println("Invalid choice!");
            continue;
        }

        int quantity = readInt("Enter quantity: ");
        if (quantity <= 0) {
            System.out.println("Quantity must be positive!");
            continue;
        }

        JuiceOrder order = new JuiceOrder(selected, quantity);
        orders.add(order);
        total += order.getTotal();

        System.out.printf("\n✓ Added: %s x %d = %d TK\n", selected.displayName, quantity, order.getTotal());
        System.out.println("Current total: " + total + " TK");
    }

    if (!orders.isEmpty()) {
        BillFileService.saveOrders(orders); // EXTRACTED to separate class
        System.out.println("\n✅ Juice/Plant Drinks Bill Generated. Total: " + total + " TK");
    }

    return total;
}
```

Benefits

- improved readability
- easier debugging
- each method performs one task

3. Duplicate Code (Dispensable)

Location: Multiple try-catch blocks for input handling

Code Snippet (Smell):

```
// First try-catch for choice
try {
    choice = Integer.parseInt(scanner.nextLine().trim());
} catch (NumberFormatException e) {
    System.out.println("Invalid input!");
    continue;
}

// Second try-catch for quantity (SAME PATTERN)
try {
    quantity = Integer.parseInt(scanner.nextLine().trim());
    if (quantity <= 0) {
        System.out.println("Quantity must be positive!");
        continue;
    }
} catch (NumberFormatException e) {
    System.out.println("Invalid quantity!");
    continue;
}
```

Problem

The same input validation code appeared multiple times.

This causes:

- code duplication
- maintenance difficulty
- violation of the DRY principle

Remedy: EXTRACT METHOD

```
private int readInt(String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            return Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Invalid input!"); // Consistent message
        }
    }
}
```

Benefits

- centralized validation logic
- reusable method
- reduced duplicate code

4. Magic String (Primitive Obsession)

Location: Hardcoded file path

Code Snippet (Smell):

```
try (FileWriter writer = new FileWriter("cafe/CafeBills/BillJuiceOrPlant.txt", true)) {  
    // Magic string directly in code  
}
```

Problem

- Hardcoded values make code harder to maintain.
- If the file path changes, it must be updated everywhere.

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {  
    public static final String BILL_DIR = "cafe/CafeBills";  
    public static final String BILL_FILE = BILL_DIR + "/BillJuiceOrPlant.txt";  
    // Used throughout the class  
}
```

Benefits

- easier maintenance
- better readability

5. Magic Strings for Menu (Primitive Obsession)

Location: Menu headers and footers hardcoded

Code Snippet (Smell):

```
System.out.println(x: "\n===== JUICE / PLANT DRINK MENU =====");  
for (int i = 0; i < JuiceItem.values().length; i++) {  
    JuiceItem juice = JuiceItem.values()[i];  
    System.out.printf(format: "%d. %s (%d Rs)\n", i+1, juice.name, juice.price);  
}  
System.out.println(x: "0. Nothing / Go Back");  
System.out.println(x: "=====");
```

Problem

Two responsibilities mixed together:

- input parsing
- validation

Remedy: REPLACE MAGIC STRING WITH CONSTANT

```
private static class Constants {  
    public static final String MENU_HEADER = "\n==== JUICE / PLANT DRINK MENU =====";  
    public static final String MENU_FOOTER = "\n0. Nothing / Go Back\n=====\\n";  
}  
}
```

Benefit

- clearer logic
- easier testing
- modular validation

6. Mixed Validation Logic (Primitive Obsession)

Location: Quantity validation inside try-catch

Code Snippet (Smell):

```
try {  
    quantity = Integer.parseInt(scanner.nextLine().trim());  
    if (quantity <= 0) { // Validation mixed with parsing  
        System.out.println("Quantity must be positive!");  
        continue;  
    }  
} catch (NumberFormatException e) {  
    System.out.println("Invalid quantity!");  
    continue;  
}
```

Remedy: SEPARATE CONCERNS

```
int quantity = readInt("Enter quantity: ");  
if (quantity <= 0) { // Validation separate  
    System.out.println("Quantity must be positive!");  
    continue;  
}
```

7. Incomplete Abstraction (Dispensable)

Location: File operations not properly abstracted

Code Snippet (Smell):

```
private void saveToFile(List<JuiceOrder> orders) {
    try {
        new File("cafe/CafeBills").mkdirs();

        try (FileWriter writer = new FileWriter("cafe/CafeBills/BillJuiceOrPlant.txt", true)) {
            for (JuiceOrder order : orders) {
                writer.write(order.getBillLine() + "\n");
            }
        }
    } catch (IOException e) {
        System.out.println("Error saving: " + e.getMessage());
    }
}
```

Problem

File operations were **not properly abstracted**.

Remedy: EXTRACT CLASS (Better Abstraction)

```
private static class BillFileService {
    static void saveOrders(List<JuiceOrder> orders) {
        try {
            new File(Constants.BILL_DIR).mkdirs();
            try (FileWriter writer = new FileWriter(Constants.BILL_FILE, true)) {
                for (JuiceOrder order : orders) {
                    writer.write(order.getBillLine() + "\n");
                }
            }
        } catch (IOException e) {
            System.out.println("Error saving: " + e.getMessage());
        }
    }
}
```

Benefit

- better abstraction
- reusable file handling
- separation of responsibilities

Food Selection:

1. Dead Code / Unused Method (Dispensable)

Location: collectBillLines() method is empty and never properly implemented

Code Snippet (Smell):

```
private void collectBillLines(FoodHandler handler) {  
    // This method would need to access the bill lines from the specific handler  
    // For now, the bill lines are written directly to files by the item classes  
}
```

Problem

This method is never used in the program and does not perform any operation.

Such unused methods increase the size of the code and create confusion for developers who try to understand the program.

Dead code causes:

- unnecessary complexity
- larger codebase
- confusion about program functionality

Remedy: REMOVE DEAD CODE

```
// Method completely removed from refactored code  
// Bill lines are now properly collected via FoodOrderResult
```

Benefit

- cleaner code structure
- easier code maintenance
- improved readability

2. Incomplete Abstraction / Lazy Class (Dispensable)

Location: NothingFoodHandler is unnecessary

Code Snippet (Smell):

```
class NothingFoodHandler implements FoodHandler {  
    public int handle(String regNumber) {  
        System.out.println("Exiting Food Menu...");  
        System.out.println("*****");  
        return 0;  
    }  
    public String getName() { return "Nothing"; }  
    public int getCode() { return 0; }  
}
```

Problem

A class that performs almost no work is considered a **Lazy Class**. Such classes increase unnecessary complexity and provide no real benefit.

Remedy: REMOVE LAZY CLASS

```
// In refactored code, '0' is handled directly in the loop without a separate handler
if (choice == 0) break; // Simple and clean
```

Benefit

- simplified class structure
- reduced unnecessary abstraction

3. Speculative Generality (Dispensable)

Location: collectBillLines() method with TODO comment

Code Snippet (Smell):

```
private void collectBillLines(FoodHandler handler) {
    // This method would need to access the bill lines from the specific handler
    // For now, the bill lines are written directly to files by the item classes
    // TODO: Implement this properly
}
```

Problem

This is called **Speculative Generality**, where code is written for features that are not currently needed.

It violates the **YAGNI principle (You Aren't Gonna Need It)**.

Problems caused:

- unnecessary code
- confusion for future developers
- harder maintenance

Remedy: REMOVE SPECULATIVE CODE / YAGNI

```
// Method removed - when needed, implement properly with FoodOrderResult
```

Benefit

- simpler and cleaner code
- reduced unnecessary complexity

4. Long Parameter List (Bloater)

Location: Observer interface has too many parameters

Code Snippet (Smell):

```
// Observer Interface
interface FoodOrderObserver {
    void onFoodOrdered(int total, String category);
}

// Logging Observer
class LoggingFoodObserver implements FoodOrderObserver {
    private int runningTotal = 0;

    @Override
    public void onFoodOrdered(int total, String category) {
        runningTotal += total;
        System.out.println("[LOG] " + category + " order: " + total + " TK (Running Total: " + runningTotal + " TK)");
    }
}
```

Problem

A long parameter list makes methods difficult to read and understand.

Problems include:

- reduced readability
- higher risk of incorrect parameter order
- harder method maintenance

Remedy: INTRODUCE PARAMETER OBJECT

```
// Observer Interface
interface FoodOrderObserver { 4 usages 1 implementation MuniraRoma
    void onFoodOrdered(int total, String category, List<String> billLines); 1 usage
}
```

Benefit

- cleaner method signatures
- improved readability
- easier future extension

5. Inconsistent Error Handling (Coupler)

Location: Mix of scanner exceptions and manual validation

Code Snippet (Smell):

```
public int readChoice() {
    System.out.print("Enter Your Choice: ");
    while (true) {
        try {
            int choice = scanner.nextInt();
            scanner.nextLine();
            return choice;
        } catch (InputMismatchException e) {
            System.out.println("Invalid input. Please enter a number.");
            scanner.nextLine();
        }
    }
}
```

Problem

Inconsistent error handling creates unpredictable behavior and messy code.

Remedy: UNIFY ERROR HANDLING

```
// Refactored code uses consistent approach
private int readIntInput(String prompt) {
    while (true) {
        System.out.print(prompt);
        try {
            return Integer.parseInt(scanner.nextLine().trim());
        } catch (NumberFormatException e) {
            System.out.println("Invalid input, enter a number.");
        }
    }
}
```

Benefit

- consistent validation
- cleaner code structure

6. Feature Envy (Coupler)

Location: FoodSelectionService holds too much state

Code Snippet (Smell):

```
class FoodSelectionService {
    private MenuDisplay menuDisplay;
    private InputReader inputReader;
    private FoodHandlerFactory handlerFactory;
    private List<FoodOrderObserver> observers;
    private int grandTotal;          // These should be local variables
    private List<String> allOrderLines; // Not method parameters
}
```

Problem

This class was trying to control too much functionality. Feature Envy occurs when a class **uses data that belongs to other classes more than its own data**.

Remedy: MOVE METHOD / LOCALIZE VARIABLES

```
// Refactored code makes grandTotal and billLines local to method
public int selectFood(String regNumber) {
    int grandTotal = 0; // Local variable
    List<String> allBillLines = new ArrayList<>(); // Local variable

    // ... processing
    return grandTotal;
}
```

Benefit

- better class responsibility
- reduced coupling

7. Primitive Obsession (Bloater)

Location: Using int codes for food categories

Code Snippet (Smell):

```
class FastFoodHandler implements FoodHandler {
    public int getCode() { return 1; } // Magic number
}

class DesiFoodHandler implements FoodHandler {
    public int getCode() { return 2; } // Magic number
}
```

Problem

Using primitive values to represent domain concepts reduces clarity.

Remedy: REPLACE MAGIC NUMBER WITH ENUM

```
// Refactored code uses enum
enum FoodCategory {
    FAST_FOOD("Fast Food"),
    DESI_FOOD("Desi Food");

    public static FoodCategory fromCode(int code) {
        if (code == 1) return FAST_FOOD;
        else if (code == 2) return DESI_FOOD;
        else return null;
    }
}
```

Benefit

- better readability
- type safety
- improved design

8. Switch-if Statements (OO Abuser)

Location: Implicit switching on handler codes

Code Snippet (Smell):

```
FoodHandler handler = handlerFactory.getHandler(choice);

if (handler == null) {
    System.out.println(x: "Please choose a correct option.");
    System.out.println(x: "*****");
    continue;
}

if (choice == 0) {
    System.out.println(x: "Exiting Food Menu...");
    System.out.println(x: "*****");
    break;
}

int total = handler.handle(regNumber);
grandTotal += total;

if (total > 0) {
    notifyObservers(total, handler.getName());
    // Collect bill lines from the handler's class
    collectBillLines(handler);
}
}

if (grandTotal > 0) {
    System.out.println(x: "\n=== Food Section Summary ===");
    System.out.println("Total Food Bill: " + grandTotal + " TK");
    System.out.println(x: "*****");
}

return grandTotal;
}
```

Problem

Large conditional blocks violate object-oriented design principles.

Remedy: REPLACE CONDITIONAL WITH POLYMORPHISM

```
// Factory for food handlers
class FoodHandlerFactory { 2 usages  MuniraRoma
    private final Map<FoodCategory, FoodHandler> handlers = new HashMap<>(); 3 usages

    public FoodHandlerFactory() { 1 usage  MuniraRoma
        handlers.put(FoodCategory.FAST_FOOD, new FastFoodHandler());
        handlers.put(FoodCategory.DESI_FOOD, new DesiFoodHandler());
    }

    public FoodHandler getHandler(FoodCategory category) { return handlers.get(category); }
}
```

Benefit

- cleaner design
- easier to extend new food categories

Drink Selection:

1. Dead Code / Unused Class (Dispensable)

Location: DrinkSelectionResult class is defined but never used

Code Snippet (Smell):

```
// Result Class for Drink Selection - NEVER USED!
class DrinkSelectionResult {
    private final Map<DrinkCategory, Integer> categoryTotals;
    private final int grandTotal;

    public DrinkSelectionResult(Map<DrinkCategory, Integer> categoryTotals, int grandTotal) {
        this.categoryTotals = new LinkedHashMap<>(categoryTotals);
        this.grandTotal = grandTotal;
    }

    // ... methods like printSummary(), hasOrders(), etc.
    // BUT THIS CLASS IS NEVER INSTANTIATED OR USED ANYWHERE!
}
```

Problem

Unused classes increase code complexity and confusion.

Remedy: REMOVE DEAD CODE

```
// Removed completely from refactored code - YAGNI principle
```

Benefit

- simpler project structure

2. Speculative Generality (Dispensable)

Location: DrinkMenuDisplayer interface with single implementation

Code Snippet (Smell):

```
// Menu Displayer Interface - Only one implementation
interface DrinkMenuDisplayer {
    void display();
}

class DrinkCategoryConsoleMenuDisplayer implements DrinkMenuDisplayer {
    @Override
    public void display() {
        System.out.println("\n==== DRINK CATEGORIES =====");
        System.out.println("1. Soft Drinks");
        System.out.println("2. Coffee");
        System.out.println("3. Juice or Plant Drinks");
        System.out.println("0. Nothing / Go Back");
        System.out.println("=====");
    }
}
```

Problem

Creating interfaces without multiple implementations leads to unnecessary abstraction.

Remedy: REMOVE UNNECESSARY INTERFACE

```
// Refactored code uses simple method instead of interface
private void displayMenu() {
    System.out.println("\n==== DRINK CATEGORIES =====");
    System.out.println("1. Soft Drinks");
    System.out.println("2. Coffee");
    System.out.println("3. Juice/Plant Drinks");
    System.out.println("0. Nothing / Go Back");
    System.out.println("=====");
}
```

Benefit

- simpler architecture

3. Speculative Generality - Input Reader (Dispensable)

Location: DrinkInputReader interface with single implementation

Code Snippet (Smell):

```
// Input Reader Interface - Only one implementation
interface DrinkInputReader {
    int readInt();
}

class DrinkConsoleInputReader implements DrinkInputReader {
    private final Scanner scanner;

    @Override
    public int readInt() {
        while (true) {
            try {
                System.out.print("Enter Your Choice: ");
                int val = scanner.nextInt();
                scanner.nextLine();
                return val;
            } catch (InputMismatchException e) {
                System.out.println("Invalid input! Please enter a number.");
                scanner.nextLine();
            }
        }
    }
}
```

Problem

Creating interfaces without multiple implementations leads to unnecessary abstraction.

Remedy: REMOVE UNNECESSARY INTERFACE

```
private void displayMenu() {
    System.out.println("\n==== DRINK CATEGORIES =====");
    System.out.println("1. Soft Drinks");
    System.out.println("2. Coffee");
    System.out.println("3. Juice/Plant Drinks");
    System.out.println("0. Nothing / Go Back");
    System.out.println("=====");
}
```

4. Service Factory Pattern Overuse (Speculative Generality)

Location: DrinkSelectionServiceFactory is overkill

Code Snippet (Smell):

```
// Service Factory
class DrinkSelectionServiceFactory {
    private static DrinkSelectionService defaultService;

    public static DrinkSelectionService createDefaultService() {
        if (defaultService == null) {
            defaultService = new DrinkSelectionService();
            defaultService.addObserver(new LoggingDrinkObserver());
        }
        return defaultService;
    }

    public static DrinkSelectionService createFreshService() {
        DrinkSelectionService service = new DrinkSelectionService();
        service.addObserver(new LoggingDrinkObserver());
        return service;
    }
}
```

Problem

Factory pattern added unnecessary complexity.

Remedy: SIMPLIFY / REMOVE FACTORY

```
public class DrinkSelection {
    private static final Scanner scanner = new Scanner(System.in);

    public static void myDrink(String regNumber) {
        DrinkSelectionService service = new DrinkSelectionService(scanner);
        service.addObserver(new LoggingDrinkObserver());
        service.selectDrink(regNumber);
    }
}
```

Benefit

- simpler code
- easier understanding

5. Feature Envy / Inappropriate Intimacy (Coupler)

Location: DrinkSelectionService holds grandTotal as field but should be local

Code Snippet (Smell):

```
class DrinkSelectionResult {
    private final Map<DrinkCategory, Integer> categoryTotals;
    private final int grandTotal;

    public DrinkSelectionResult(Map<DrinkCategory, Integer> categoryTotals, int grandTotal) {
        this.categoryTotals = new LinkedHashMap<>(categoryTotals);
        this.grandTotal = grandTotal;
    }

    public Map<DrinkCategory, Integer> getCategoryTotals() {
        return Collections.unmodifiableMap(categoryTotals);
    }

    public int getGrandTotal() {
        return grandTotal;
    }

    public boolean hasOrders() {
        return grandTotal > 0;
    }
}
```

Problem

This value should be temporary and local to the method.

Remedy: LOCALIZE VARIABLE

```
public int selectDrink(String regNumber) {
    grandTotal = 0;
    while (true) {
        displayMenu();
        int choice = readChoice();

        if (choice == 0) break;

        Optional<DrinkHandler> handlerOpt = factory.getHandler(choice);
        if (handlerOpt.isPresent()) {
            DrinkHandler handler = handlerOpt.get();
            int total = handler.handle(regNumber);
            grandTotal += total;

            if (total > 0)
                notifyObservers(total, handler.getCategory().getDisplayName());
        } else {
            System.out.println("Please select a valid option!");
        }
    }

    System.out.println("Total Drinks Bill: " + grandTotal + " TK");
    return grandTotal;
}
```

Benefit

- reduced class state
- safer code

6. Duplicate Code - Multiple Service Creation Methods

Location: myDrink() and myDrinkWithTotal() are almost identical

Code Snippet (Smell):

```
public static void myDrink(String regNumber) {
    service.selectDrink(regNumber);
}

/**
 * Returns the total drinks bill amount
 */
public static int getDrinkTotal() {
    return service.getGrandTotal();
}

/**
 * Enhanced version that returns total bill amount
 */
public static int myDrinkWithTotal(String regNumber) {
    DrinkSelectionService freshService = DrinkSelectionServiceFactory.createFreshService();
    int total = freshService.selectDrink(regNumber);
    return total;
}
```

Problem

Duplicate methods increase maintenance effort.

Remedy: UNIFY METHODS

```
public static void myDrink(String regNumber) {
    DrinkSelectionService service = new DrinkSelectionService(scanner);
    service.addObserver(new LoggingDrinkObserver());
    service.selectDrink(regNumber);
}
```

Benefit

- reduced duplication
- easier maintenance

7. Lazy Class - Nothing Handler (Dispensable)

Location: NothingDrinkHandler does very little

Code Snippet (Smell):

```
class NothingDrinkHandler implements DrinkHandler {  
    @Override  
    public int handle(String regNumber) {  
        System.out.println("Exiting Drinks Menu...");  
        System.out.println("*****");  
        return 0;  
    }  
  
    @Override  
    public DrinkCategory getCategory() {  
        return DrinkCategory.NOTHING;  
    }  
}
```

Problem

Creating a separate class for a trivial operation increases complexity.

Remedy: HANDLE '0' DIRECTLY IN LOOP

```
public int selectDrink(String regNumber) {  
    grandTotal = 0;  
    while (true) {  
        displayMenu();  
        int choice = readChoice();  
  
        if (choice == 0) break;  
  
        Optional<DrinkHandler> handlerOpt = factory.getHandler(choice);  
        // ...  
    }  
}
```

Benefit

- fewer unnecessary classes
- simpler control flow

Before Refactoring:

```
-----  
Date: Fri Mar 13 19:14:42 GMT+06:00 2026  
Name: MARZIA HOSSAN  
Student ID: SP19-BCS-127  
-----Your Bill Is-----  
  
Items           Qty    Price  
Zinger Burger   2      500  
Zinger Burger   1      250  
Aloo Ka Paratha 1      100  
-----  
Total Bill:           850 TK  
Remaining Balance:    1150 TK  
-----  
Remaining Balance: 1150 TK
```

After Refactoring:

```
-----  
Date: Fri Mar 13 19:10:19 GMT+06:00 2026  
Name: QASIM ALI  
Student ID: FA18-BCS-131  
-----Your Bill Is-----  
  
Items           Qty    Price  
Zinger Burger   2      500  
Haleem          1      120  
-----  
Total Bill:           620 TK  
Remaining Balance:    1380 TK  
-----  
Remaining Balance: 1380 TK
```

3. Café Management: (Design Patterns)

1. Singleton Pattern

Files: DrinkSelection.java, FoodSelection.java

Why use it: Ensures only one instance of a class exists throughout the application, providing a global access point. Useful for shared resources like configuration managers, loggers, or scanner instances.

Code Snippet from DrinkSelection.java:

```
java

// Singleton Scanner Manager
class ScannerManager {
    private static ScannerManager instance;
    private Scanner scanner;

    private ScannerManager() {
        scanner = new Scanner(System.in);
    }

    public static ScannerManager getInstance() {
        if (instance == null) {
            instance = new ScannerManager();
        }
        return instance;
    }

    public Scanner getScanner() {
        return scanner;
    }
}
```

Code Snippet from FoodSelection.java:

```
java

// Singleton Pattern for the main FoodSelection class
public class FoodSelection {
```

```

private static FoodSelection instance;

private FoodSelection() {
    manager = new FoodSelectionManager(scanner);
}

public static synchronized FoodSelection getInstance() {
    if (instance == null) {
        instance = new FoodSelection();
    }
    return instance;
}
}

```

2. Strategy Pattern

Files: FoodSelection.java, MyFastFood.java, MyCoffee.java

Why use it: Allows selecting different algorithms (billing strategies) at runtime without changing the context class. Makes the system flexible and open for extension but closed for modification.

Code Snippet from FoodSelection.java:

```

java

// Strategy Interface
interface FoodBillingStrategy {
    OrderResult calculateBill(String regNumber);
}

// Concrete Strategy 1: Fast Food
class FastFoodBillingStrategy implements FoodBillingStrategy {
    @Override
    public OrderResult calculateBill(String regNumber) {
        MyFastFood fastFood = new MyFastFood();
        int total = fastFood.FastfoodBill();
        return new OrderResult(total, billLines, "Fast Food");
    }
}

```

```
}
```

```
// Concrete Strategy 2: Desi Food
```

```
class DesiFoodBillingStrategy implements FoodBillingStrategy {  
    @Override  
    public OrderResult calculateBill(String regNumber) {  
        MyDesiFood desiFood = new MyDesiFood();  
        int total = desiFood.DesifoodBill();  
        return new OrderResult(total, billLines, "Desi Food");  
    }  
}
```

```
// Context class that uses the strategy
```

```
class FoodOrderContext {  
    private FoodBillingStrategy strategy;  
  
    public void setStrategy(FoodBillingStrategy strategy) {  
        this.strategy = strategy;  
    }  
  
    public OrderResult executeStrategy(String regNumber) {  
        return strategy.calculateBill(regNumber);  
    }  
}
```

Code Snippet from `MyFastFood.java`:

```
java
```

```
// Strategy Interface
```

```
interface BillSavingStrategy {  
    void saveBill(List<OrderedItem> orders, int total);  
}
```

```
// Concrete Strategy 1: File Saving
```

```
static class FileSavingStrategy implements BillSavingStrategy {  
    @Override  
    public void saveBill(List<OrderedItem> orders, int total) {  
// File saving implementation  
    }  
}
```

```
}
```

```
// Concrete Strategy 2: Console Only
```

```
static class ConsoleOnlyStrategy implements BillSavingStrategy {
```

```
    @Override
```

```
    public void saveBill(List<OrderedItem> orders, int total) {
```

```
// Console only implementation
```

```
    }
```

```
}
```

Code Snippet from `MyCoffee.java`:

```
java
```

```
// Strategy Interface
```

```
interface FileSaveStrategy {
```

```
    void save(List<CoffeeOrder> orders);
```

```
}
```

```
// Concrete Strategy 1: Text File
```

```
static class TextFileSaveStrategy implements FileSaveStrategy {
```

```
    @Override
```

```
    public void save(List<CoffeeOrder> orders) {
```

```
// Text file saving
```

```
    }
```

```
}
```

```
// Concrete Strategy 2: CSV Format
```

```
static class CSVFileSaveStrategy implements FileSaveStrategy {
```

```
    @Override
```

```
    public void save(List<CoffeeOrder> orders) {
```

```
// CSV saving
```

```
    }
```

```
}
```

3. Facade Pattern

Files: `MyFinalBill.java`, `MySoftDrinks.java`, `MyJuiceOrPlantDrink.java`

Why use it: Provides a simplified interface to a complex subsystem. Hides the complexity of file operations, bill generation, and data processing behind a simple method call.

Code Snippet from `MyFinalBill.java`:

```
java

// FACADE - Simplified interface for the entire system
class BillSystemFacade {
    private final SimpleBillReader billReader;
    private final SimpleStudentReader studentReader;

    public BillSystemFacade() {
        this.billReader = new SimpleBillReader();
        this.studentReader = new SimpleStudentReader();
    }

    // Single method that handles everything
    public BillResult generateBill(String regNumber, int balance) {
        StudentInfo student = studentReader.getStudent(regNumber);
        List<BillItem> items = billReader.readAllBills();
        int total = 0;
        for (BillItem item : items) total += item.price;

        if (balance < total) {
            return BillResult.insufficientBalance(balance, total);
        }

        int remaining = balance - total;
        printBill(student, items, total, remaining);
        saveBill(student, items, total);
        billReader.clearAllBills();

        return BillResult.success(remaining, total, items.size());
    }
}
```

```
}
```

Code Snippet from MySoftDrinks.java:

```
java
```

```
// FACADE PATTERN: Provides a simplified interface
```

```
private static class SoftDrinkBillingFacade {
```

```
    public int processBill(String regNumber) {
```

```
        SoftDrinkMenu menu = new SoftDrinkMenu();
```

```
        OrderProcessor orderProcessor = new OrderProcessor();
```

```
        BillSaver billSaver = new BillSaver();
```

```
        menu.display();
```

```
        List<SoftDrinkOrder> orders = orderProcessor.takeOrders();
```

```
        if (!orders.isEmpty()) {
```

```
            return billSaver.saveOrders(orders, regNumber);
```

```
        }
```

```
        return 0;
```

```
    }
```

```
}
```

Code Snippet from MyJuiceOrPlantDrink.java:

```
java
```

```
// Facade class that simplifies file operations
```

```
private static class BillFacade {
```

```
// Simple one-method interface for saving bills
```

```
    public static void saveBill(List<JuiceOrder> orders) {
```

```
// The facade handles all the complexity internally
```

```
        FileOperator fileOp = new FileOperator();
```

```
        DirectoryManager dirManager = new DirectoryManager();
```

```
        BillFormatter formatter = new BillFormatter();
```

```
        dirManager.ensureDirectoryExists(Constants.BILL_DIR);
```

```
        String billContent = formatter.formatOrders(orders);
```

```
        fileOp.appendToFile(Constants.BILL_FILE, billContent);
```

```
    }
```

```
}
```

4. Observer Pattern

Files: DrinkSelection.java, FoodSelection.java

Why use it: Allows objects to be notified automatically when events occur (like orders being placed). Decouples the subject (order service) from the observers (logging service).

Code Snippet from DrinkSelection.java:

```
java
```

```
// Observer Interface
```

```
interface DrinkOrderObserver {  
    void onDrinkOrdered(int total, String category);  
}
```

```
// Concrete Observer: Logging
```

```
class LoggingDrinkObserver implements DrinkOrderObserver {  
    private static LoggingDrinkObserver instance;  
    private int runningTotal = 0;
```

```
@Override
```

```
public void onDrinkOrdered(int total, String category) {  
    runningTotal += total;  
    System.out.println("[LOG] " + category + " order: " +  
        total + " TK (Running Total: " + runningTotal + " TK)");  
}  
}
```

```
// Subject/Observable class
```

```
class DrinkSelectionService {  
    private final List<DrinkOrderObserver> observers = new ArrayList<>();  
  
    public void addObserver(DrinkOrderObserver observer) {  
        observers.add(observer);  
    }  
}
```

```

private void notifyObservers(int total, String category) {
    for (DrinkOrderObserver obs : observers) {
        obs.onDrinkOrdered(total, category);
    }
}
}

```

Code Snippet from FoodSelection.java:

```
java
```

```
// Observer Pattern for Logging
```

```

interface OrderObserver {
    void onOrderPlaced(OrderResult result);
}

```

```

class LoggingObserver implements OrderObserver {
    @Override
    public void onOrderPlaced(OrderResult result) {
        System.out.println("[LOG] " + result.getCategory() + " order: " +
            result.getTotal() + " TK (Running Total: " +
            result.getRunningTotal() + " TK)");
    }
}

```

```

class FoodSelectionManager {
    private final List<OrderObserver> observers = new ArrayList<>();

```

```

    public void addObserver(OrderObserver observer) {
        observers.add(observer);
    }

```

```

    private void notifyObservers(OrderResult result) {
        for (OrderObserver observer : observers) {
            observer.onOrderPlaced(result);
        }
    }
}

```

5. Factory Pattern

Files: DrinkSelection.java, FoodSelection.java

Why use it: Centralizes object creation logic, making it easy to add new types without modifying existing code. Provides a clean way to get appropriate handlers/strategies.

Code Snippet from DrinkSelection.java:

```
java

// SINGLETON: Drink Handler Factory
class DrinkHandlerFactory {
    private static DrinkHandlerFactory instance;
    private final Map<Integer, DrinkHandler> handlers = new HashMap<>();

    private DrinkHandlerFactory(Scanner scanner) {
        handlers.put(1, new SoftDrinksHandler(scanner));
        handlers.put(2, new CoffeeHandler(scanner));
        handlers.put(3, new JuicePlantHandler(scanner));
        handlers.put(0, new NothingDrinkHandler());
    }

    public static DrinkHandlerFactory getInstance() {
        if (instance == null) {
            instance = new DrinkHandlerFactory(ScannerManager.getInstance().getScanner());
        }
        return instance;
    }

    public Optional<DrinkHandler> getHandler(int code) {
        return Optional.ofNullable(handlers.get(code));
    }
}
```

Code Snippet from FoodSelection.java:

```
java

// Strategy Factory
class StrategyFactory {
```

```

private static final Map<Integer, FoodBillingStrategy> strategies = new HashMap<>();

static {
    strategies.put(1, new FastFoodBillingStrategy());
    strategies.put(2, new DesiFoodBillingStrategy());
}

public static FoodBillingStrategy getStrategy(int choice) {
    return strategies.get(choice);
}

public static boolean isValidChoice(int choice) {
    return strategies.containsKey(choice);
}
}

```

6. Builder Pattern (Implicit/Bill Formatting)

Files: `MyJuiceOrPlantDrink.java`, `MyFinalBill.java`

Why use it: Constructs complex objects (bill strings) step by step. Provides a clean way to build formatted bill content without messy string concatenation.

Code Snippet from `MyJuiceOrPlantDrink.java`:

```

java

// BillFormatter acts as a Builder for bill content
private static class BillFormatter {

    public String formatOrders(List<JuiceOrder> orders) {
        StringBuilder sb = new StringBuilder(); // Builder for string
        sb.append("=== JUICE/PLANT DRINK BILL ===\n");
        sb.append(String.format("%-25s %3s %8s\n", "Item", "Qty", "Total(TK)"));
        sb.append("-----\n");

        for (JuiceOrder order : orders) {
            sb.append(order.getBillLine()).append("\n");
        }
    }
}

```

```

}

int grandTotal = orders.stream().mapToInt(JuiceOrder::getTotal).sum();
sb.append("-----\n");
sb.append(String.format("%-25s %15d TK\n", "GRAND TOTAL:", grandTotal));
sb.append("=====\n\n");

return sb.toString();
}
}

```

7. Enum Singleton Pattern

Files: `DrinkSelection.java`

Why use it: Provides a more robust singleton implementation that is inherently serialization-safe and thread-safe.

Code Snippet from `DrinkSelection.java`:

```

java

// ENUM for Drink Categories - Acts as a type-safe singleton
enum DrinkCategory {
    SOFT_DRINKS("Soft Drinks", 1),
    COFFEE("Coffee", 2),
    JUICE_PLANT("Juice/Plant Drinks", 3),
    NOTHING("Nothing / Go Back", 0);

    private final String displayName;
    private final int code;

    DrinkCategory(String displayName, int code) {
        this.displayName = displayName;
        this.code = code;
    }

    // Enum provides built-in singleton behavior
    public static Optional<DrinkCategory> fromCode(int code) {

```

```

return Arrays.stream(values())
    .filter(c -> c.code == code)
    .findFirst();
}
}

```

8. Chain of Responsibility Pattern

Files: DrinkSelection.java (Menu options handling)

Why use it: Allows multiple handlers to process a request without the sender knowing which handler will process it. Each menu option routes to a different handler.

Code Snippet from DrinkSelection.java:

```

java

// Factory maps choices to handlers (implicit chain)
class DrinkHandlerFactory {
    private final Map<Integer, DrinkHandler> handlers = new HashMap<>();

    private DrinkHandlerFactory(Scanner scanner) {
        handlers.put(1, new SoftDrinksHandler(scanner)); // Link 1 → Handler
        handlers.put(2, new CoffeeHandler(scanner));    // Link 2 → Handler
        handlers.put(3, new JuicePlantHandler(scanner)); // Link 3 → Handler
        handlers.put(0, new NothingDrinkHandler());    // Link 0 → Handler
    }

    public Optional<DrinkHandler> getHandler(int code) {
        return Optional.ofNullable(handlers.get(code));
    }
}

// Usage in service
public int selectDrink(String regNumber) {
    while (true) {
        int choice = readChoice();
        if (choice == 0) break;
    }
}

```

```
// Chain of responsibility - handler resolves the request  
Optional<DrinkHandler> handlerOpt = factory.getHandler(choice);  
if (handlerOpt.isPresent()) {  
    int total = handlerOpt.get().handle(regNumber);  
    grandTotal += total;  
}  
}  
}
```

Before Design Pattern:

```
-----
Date: Fri Mar 13 19:14:42 GMT+06:00 2026
Name: MARZIA HOSSAN
Student ID: SP19-BCS-127
-----Your Bill Is-----

Items          Qty    Price
Zinger Burger    2      500
Zinger Burger    1      250
Aloo Ka Paratha  1      100
-----
Total Bill:          850 TK
Remaining Balance:   1150 TK
-----
Remaining Balance: 1150 TK
```

After Design Pattern :

```
-----
Date: Fri Mar 13 19:10:19 GMT+06:00 2026
Name: QASIM ALI
Student ID: FA18-BCS-131
-----Your Bill Is-----

Items          Qty    Price
Zinger Burger    2      500
Haleem           1      120
-----
Total Bill:          620 TK
Remaining Balance:   1380 TK
-----
Remaining Balance: 1380 TK
```

University Management System:

Name	ID	Contriution
Muziba Mahmuda Mithy	2022-1-60-011	Course Management
Tabassum Nahar Yeah	2023-1-60-068	Transport Management
Sirajam Munira Naba	2022-2-60-021	Library Management
Wasima Khanom Munira	2022-1-60-287	Café Management